

Технічне завдання

Проект: Консольний музичний плеєр у стилі Spotify

Мова програмування: C++

Парадигма: ООП

Тип програми: консольна, зі стильним кольоровим оформленням

1. Мета проєкту

Створити консольну програму для роботи з локальною музичною бібліотекою, яка дозволяє додавати треки, переглядати список пісень, шукати їх, створювати плейлисти та керувати відтворенням через меню.

Інтерфейс має бути не просто “чорне вікно з текстом”, а акуратний, кольоровий і приємний візуально.

2. Призначення програми

Програма призначена для ефективного управління локальною музичною бібліотекою з підтримкою пошуку, плейлистів та черги відтворення.

Реалізує спрощену модель музичного плеєра з консольним інтерфейсом та структурованою логікою взаємодії.

3. Основний функціонал

Програма повинна дозволяти:

- додавати треки до бібліотеки;
- шукати трек за назвою;
- створювати плейлисти;
- додавати треки у плейлист;
- переглядати вміст плейлиста;
- формувати чергу відтворення;
- перемикаати поточний трек на наступний або попередній;
- позначати треки як улюблені;
- зберігати дані у файл і завантажувати їх при запуску програми.

4. Особливості інтерфейсу

Консольний інтерфейс має бути стильним і зручним:

- кольорове меню;
- виділення заголовків, помилок, успішних дій і активного треку різними кольорами;
- зрозуміла нумерація пунктів меню;
- охайне форматування списків;
- окремий блок для поточного треку, черги та улюблених.

Для оформлення можна використати ANSI-кольори або можливості Windows-консолі.

5. Обмеження проєкту

Щоб проєкт був реалістичним для виконання, у базовій версії:

- обов'язково реалізовувати справжнє відтворення музики;
- можна зробити **імітацію плеєра**: статус "Playing / Paused / Stopped" і перемикання між треками;
- не потрібно робити графіку, обкладинки, повзунки, кнопки мишкою;
- достатньо роботи через клавіатуру і консольне меню.

6. Основні класи

Track

Містить інформацію про трек:

- назва;
- виконавець;
- тривалість;
- шлях до файлу;
- статус "улюблений".

Playlist

Містить:

- назву плейлиста;
- список треків.

Методи:

- додати трек;
- видалити трек;
- показати треки.

Queue

Містить чергу відтворення.

Методи:

- додати в чергу;
- видалити з черги;
- отримати наступний трек;
- очистити чергу.

MusicLibrary

Містить:

- усю бібліотеку треків;
- список плейлистів;
- список улюблених.

Методи:

- додавання треків;
- пошук;
- створення плейлистів;
- збереження та завантаження даних.

Player

Керує логікою плеєра:

- play();
- pause();
- stop();
- next();
- previous();
- shuffle().

7. Принципи ООП

У проєкті мають використовуватися:

- **інкапсуляція** — закриті поля та робота через методи;
- **абстракція** — поділ програми на логічні класи;
- **поліморфізм** — за можливості для розширення логіки;
- **наслідування** — не обов'язково, але можна використати, якщо буде доречно.

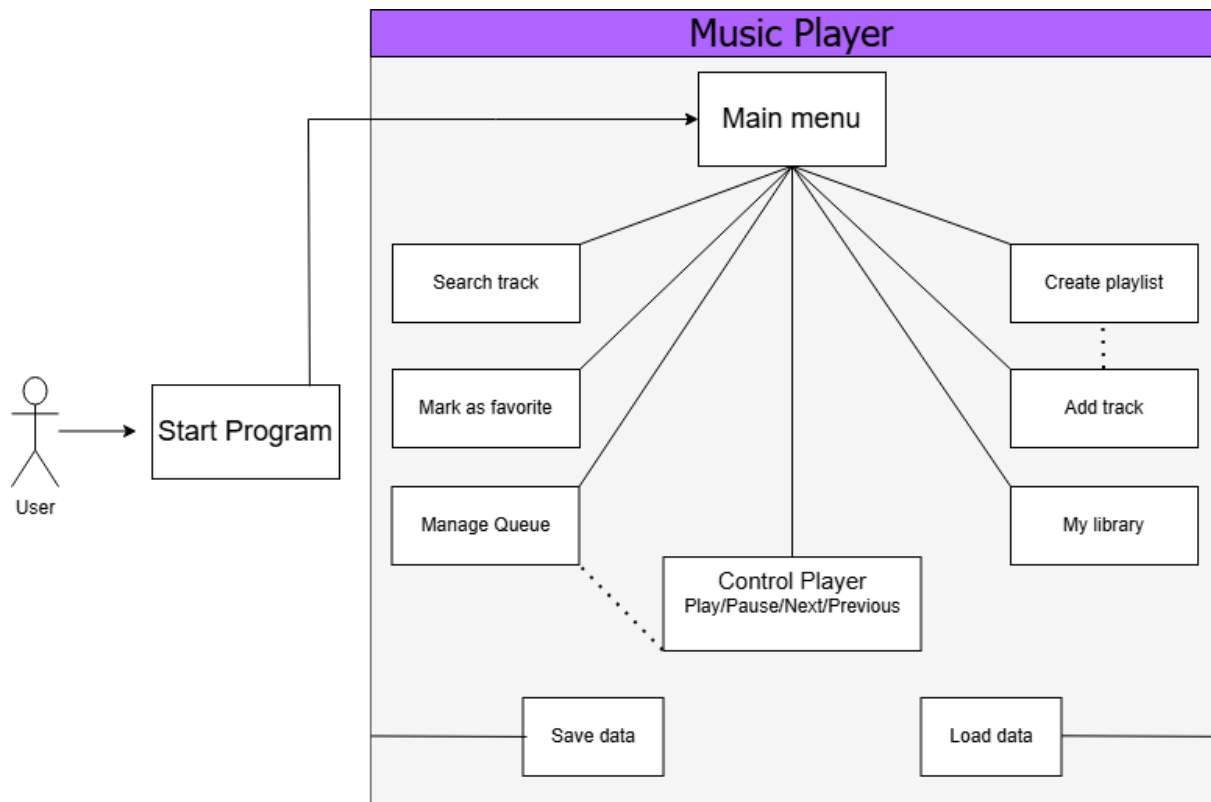
8. Збереження даних

Програма повинна зберігати у файл:

- список треків;
- плейлисти;
- улюблені треки;
- останню чергу.

Формат збереження: **JSON**.

9. Сценарій роботи користувача



10. Очікуваний результат

У результаті має бути створена **стильна консольна програма на C++**, яка:

- має красивий кольоровий інтерфейс;
- демонструє ООП;
- дозволяє працювати з музичною бібліотекою;
- підтримує плейлисти, пошук, чергу та улюблені;
- виглядає акуратно і реально може бути захищена як навчальний проєкт.

11. Що робить проєкт більш реальним

У цій версії ми прибрати занадто важкі речі:

- Qt-інтерфейс;
- обкладинки;
- повзунки;
- складне мультимедіа;
- перевантажений функціонал.

Зате залишили те, що реально зробити добре:

- сильну структуру класів;
- гарне консольне меню;
- кольорове оформлення;
- персистентне збереження даних у форматі `.json` зі структурованим представленням
- логіку музичного плеєра.